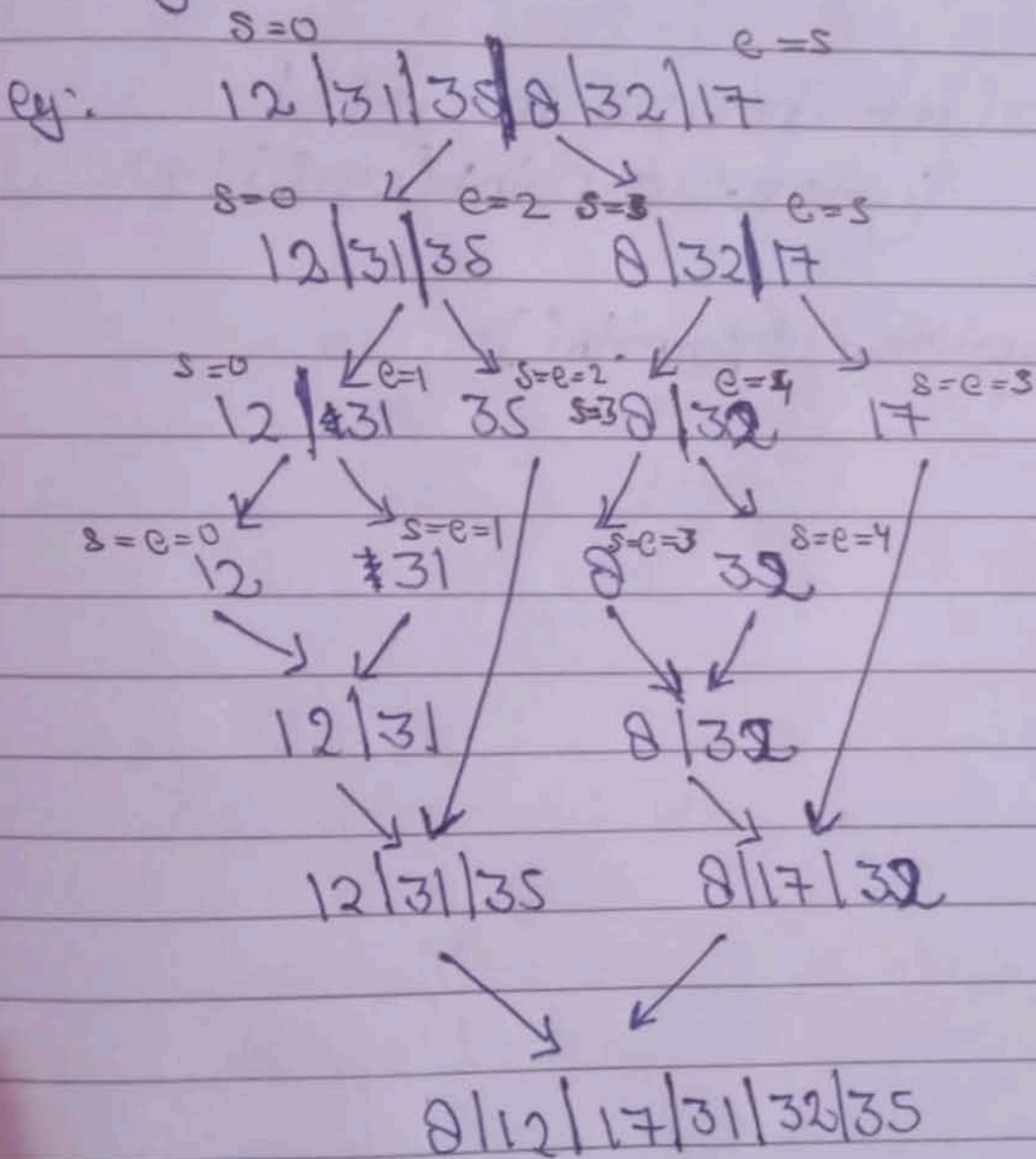


Merge Sort

① Divide and conquer
→ mid

② Merge parts to create a sorted array



Module-Week-3

1) Merge Sort (follows Divide and Conquer technique)

- Logic:
1. Divide the array into two halves
 2. Continue dividing until each subarray contains only one element.
 3. Merge the subarrays in sorted order.
 4. Continue merging until the complete array is sorted

Time Complexity

Best: $O(n \log n)$, Average: $O(n \log n)$, Worst: $O(n \log n)$

Space: $O(n)$, stable

Code

```
import java.util.Scanner;
```

```
public class Merge {
```

```
    static void mergeSort(int[] arr, int left, int right) {
```

```
        if (left < right) {
```

```
            int mid = (left + right) / 2;
```

```
            mergeSort(arr, left, mid);
```

```
            mergeSort(arr, mid + 1, right);
```

```
            merge(arr, left, mid, right);
```

```
        }
```

```
    }
```

```
    static void merge(int[] arr, int left, int mid, int right) {
```

```
        int n1 = mid - left + 1;
```

```
        int n2 = right - mid;
```

```
        int[] L = new int[n1];
```

```
        int[] R = new int[n2];
```

```
        for (int i = 0; i < n1; i++)
```

```
            L[i] = arr[left + i];
```

```
        for (int j = 0; j < n2; j++)
```

```
            R[j] = arr[mid + 1 + j];
```

```
        int i = 0; j = 0; k = left;
```



```

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

```

```

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

```

```

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    int[] arr = new int[N];
    for (int i = 0; i < N; i++) {
        arr[i] = sc.nextInt();
    }
    mergeSort(arr, 0, N-1);
    for (int i = 0; i < N; i++) {
        System.out.print(arr[i] + " ");
    }
    sc.close();
}

```

for merge sort 2

additional code

```

// Print sorted array
for (int i = 0; i < n; i++) {
    System.out.print(arr[i] + " ");
}
System.out.println();

```

```

// Find median
int median;
if (n % 2 == 1) {
    median = arr[n/2];
} else {
    median = (arr[n/2-1] + arr[n/2]) / 2;
}

```

```

// Count = 0;
for (int i = 0; i < n; i++) {
    if (arr[i] > median) {
        Count++;
    }
}

```

```

// difference b/w max and min
int difference = arr[n-1] - arr[0];
System.out.println("Median: " + median);
System.out.println("Orders Above Median: " + Count);
System.out.println("Difference: " + difference);
sc.close();
}

```

Input: 7

12 25 18 30 15 22 40

Output: 12 15 18 22 25 30 40

Median: 22

Orders Above Median: 3

Difference: 28

Input

6 450 1200 800 650 300 1500

Output

300 450 650 800 1200 1500

J Merge.java > Merge > mergeSort(int[], int, int)

```
1 import java.util.Scanner;
2
3 public class Merge {
4
5     static void mergeSort(int[] arr, int left, int right) {
6         if (left < right) {
7             int mid = left + (right - left) / 2;
8
9             mergeSort(arr, left, mid);
10            mergeSort(arr, mid + 1, right);
11
12            merge(arr, left, mid, right);
13        }
14    }
15
16    static void merge(int[] arr, int left, int mid, int right) {
17
18        int n1 = mid - left + 1;
19        int n2 = right - mid;
20
21        int[] L = new int[n1];
22        int[] R = new int[n2];
23
24        for (int i = 0; i < n1; i++)
25            L[i] = arr[left + i];
26
27        for (int j = 0; j < n2; j++)
28            R[j] = arr[mid + 1 + j];
29
30        int i = 0, j = 0, k = left;
31
32        while (i < n1 && j < n2) {
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java Merge
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500
PS C:\Users\Solv-IT\Desktop\java_module>
```

```

3 public class Merge {
16 static void merge(int[] arr, int left, int mid, int right) {
31
32     while (i < n1 && j < n2) {
33         if (L[i] <= R[j]) {
34             arr[k] = L[i];
35             i++;
36         } else {
37             arr[k] = R[j];
38             j++;
39         }
40         k++;
41     }
42
43     while (i < n1) {
44         arr[k] = L[i];
45         i++;
46         k++;
47     }
48
49     while (j < n2) {
50         arr[k] = R[j];
51         j++;
52         k++;
53     }
54 }
55

```

Run | Debug

```

56 public static void main(String[] args) {
57
58     Scanner sc = new Scanner(System.in);
59

```

```

6
450 1200 800 650 300 1500
300 450 650 800 1200 1500

```

PS C:\Users\Solv-IT\Desktop\java_module

J Merge.java > Merge > mergeSort(int[], int, int)

```
3 public class Merge {
16 static void merge(int[] arr, int left, int mid, int right) {
54 }
55 }
```

Run | Debug

```
56 public static void main(String[] args) {
57
58     Scanner sc = new Scanner(System.in);
59
60     int N = sc.nextInt();
61
62     int[] arr = new int[N];
63
64     for (int i = 0; i < N; i++) {
65         arr[i] = sc.nextInt();
66     }
67
68     mergeSort(arr, left: 0, N - 1);
69
70     for (int i = 0; i < N; i++) {
71         System.out.print(arr[i] + " ");
72     }
73
74     sc.close();
75 }
76 }
77 }
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java Merge
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500
PS C:\Users\Solv-IT\Desktop\java_module>
```

PS C:\Users\Solv-IT\Desktop\java_module> java Merge

6

450 1200 800 650 300 1500

300 450 650 800 1200 1500

PS C:\Users\Solv-IT\Desktop\java_module>

2 Merge Sort with Statistics

- Logic
1. Read input
 2. Apply Merge Sort
 3. Print sorted array.
 4. Find median.
 5. Count elements greater than median.
 6. Find the difference b/w maximum and minimum

Median

If odd: $\text{Median} = \text{arr}[\lfloor n/2 \rfloor]$

If Even: $\text{Median} = (\text{arr}[\lfloor n/2 - 1 \rfloor] + \text{arr}[\lfloor n/2 \rfloor]) / 2$

Difference: $\text{Difference} = \text{Maximum} - \text{minimum}$

Time Complexity

Sorting: $O(n \log n)$

Finding statistics: $O(n) + \underline{O(n \log n)}$

HeapSort.java • Merge.java •

Merge.java > ...

```
1 import java.util.Scanner;
2
3 public class Merge {
4
5     static void mergeSort(int[] arr, int left, int right) {
6         if (left < right) {
7             int mid = left + (right-left) / 2;
8
9             mergeSort(arr, left, mid);
10            mergeSort(arr, mid + 1, right);
11
12            merge(arr, left, mid, right);
13        }
14    }
15
16    static void merge(int[] arr, int left, int mid, int right) {
17
18        int n1 = mid - left + 1;
19        int n2 = right - mid;
20
21        int[] L = new int[n1];
22        int[] R = new int[n2];
23
24        for (int i = 0; i < n1; i++)
25            L[i] = arr[left + i];
26
27        for (int j = 0; j < n2; j++)
28            R[j] = arr[mid + 1 + j];
29
30        int i = 0, j = 0, k = left;
31
32        while (i < n1 && j < n2) {
```

powerShell ×

```
PS C:\Users\Solv-IT\Desktop\java_module> java Merge
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\Solv-IT\Desktop\java_module> 
```

Tab Movers FocusLn 1 Col 1Spaces: 4UTF-8CRLF{ }Java


```

3 public class Merge {
49     public static void main(String[] args) {
59
60         mergeSort(arr, left: 0, n - 1);
61
62         // Print sorted array
63         for (int i = 0; i < n; i++) {
64             System.out.print(arr[i] + " ");
65         }
66         System.out.println();
67
68         // Find median
69         int median;
70         if (n % 2 == 1) {
71             median = arr[n / 2];
72         } else {
73             median = (arr[n / 2 - 1] + arr[n / 2]) / 2;
74         }
75         // Count numbers greater than median
76         int count = 0;
77         for (int i = 0; i < n; i++) {
78             if (arr[i] > median) {
79                 count++;
80             }
81         }
82         // Difference between max and min
83         int difference = arr[n - 1] - arr[0];
84
85         System.out.println("Median: " + median);
86         System.out.println("Orders Above Median: " + count);
87         System.out.println("Difference: " + difference);
88
89         sc.close();

```

```

7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

```

○ PS C:\Users\Solv-IT\Desktop\java_module


```

J Merge.java > ...
3  public class Merge {
16      static void merge(int[] arr, int left, int mid, int right) {
31
32          while (i < n1 && j < n2) {
33              if (L[i] <= R[j]) {
34                  arr[k++] = L[i++];
35              } else {
36                  arr[k++] = R[j++];
37              }
38          }
39
40          while (i < n1) {
41              arr[k++] = L[i++];
42          }
43
44          while (j < n2) {
45              arr[k++] = R[j++];
46          }
47      }
48
49      Run | Debug
50      public static void main(String[] args) {
51
52          Scanner sc = new Scanner(System.in);
53
54          int n = sc.nextInt();
55          int[] arr = new int[n];
56
57          for (int i = 0; i < n; i++) {
58              arr[i] = sc.nextInt();
59          }
60          mergeSort(arr, left: 0, n - 1);

```

```

PS C:\Users\Solv-IT\Desktop\java_module> java Merge
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\Solv-IT\Desktop\java_module> 

```


Quick Sort

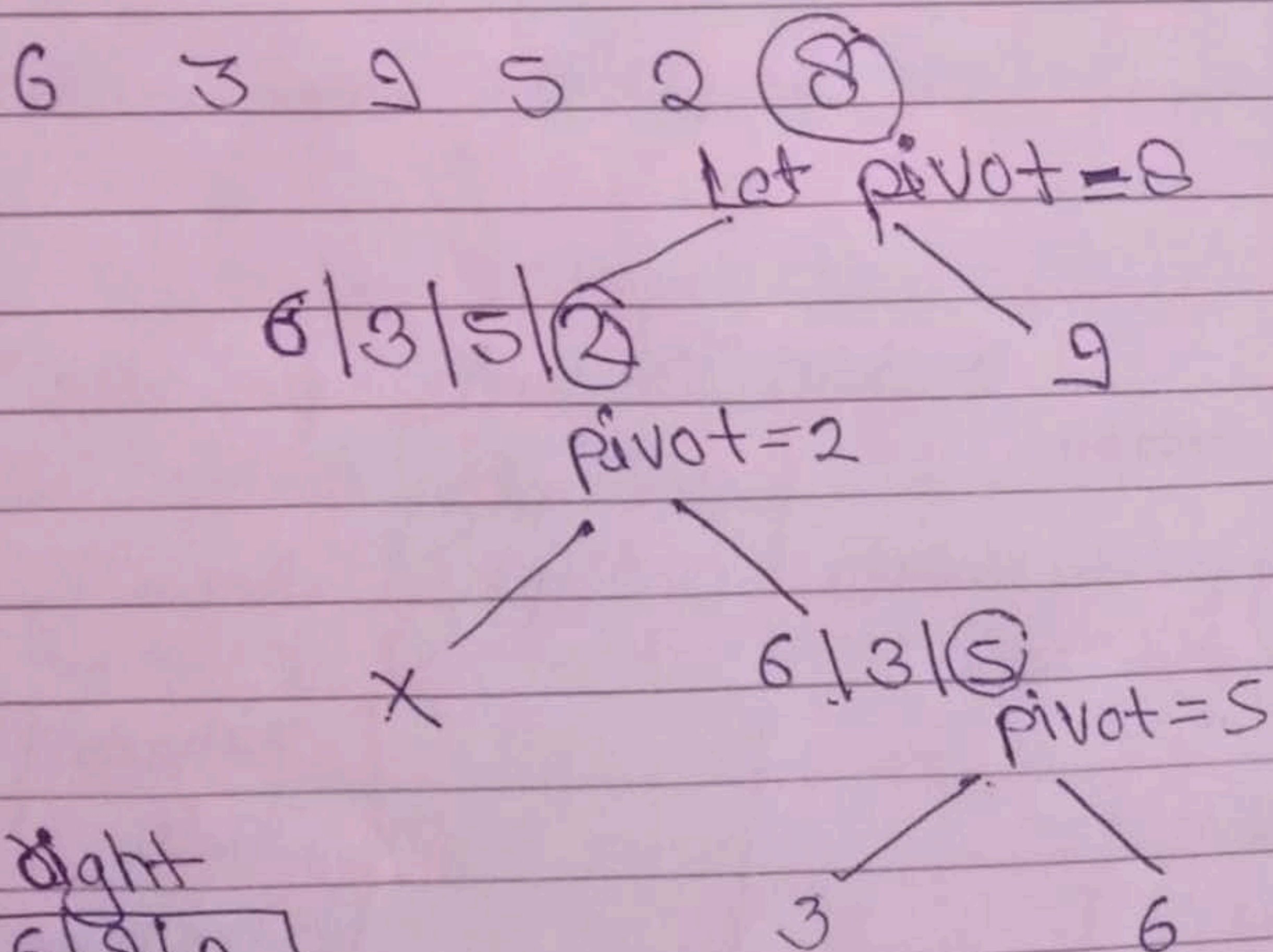
Pivot and Partition

eg:

6	3	9	5	2	8
---	---	---	---	---	---

Pivot can be taken as

- (i) random element
- (ii) average of all elements
- (iii) first element
- (iv) last element



Left to Right

2	3	5	6	8	9
---	---	---	---	---	---

Sorted array

Logic; 1. select last element as Pivot.

2. Place all smaller element on the left.

3. Place all greater elements on the right.

Partition

$\text{if } (\text{arr}[j] \leq \text{pivot})$

Swap()

Time Complexity

Best: $O(n \log n)$

Average: $O(n \log n)$

Worst: $O(n^2)$ → when array is already sorted

Space: $O(\log n)$

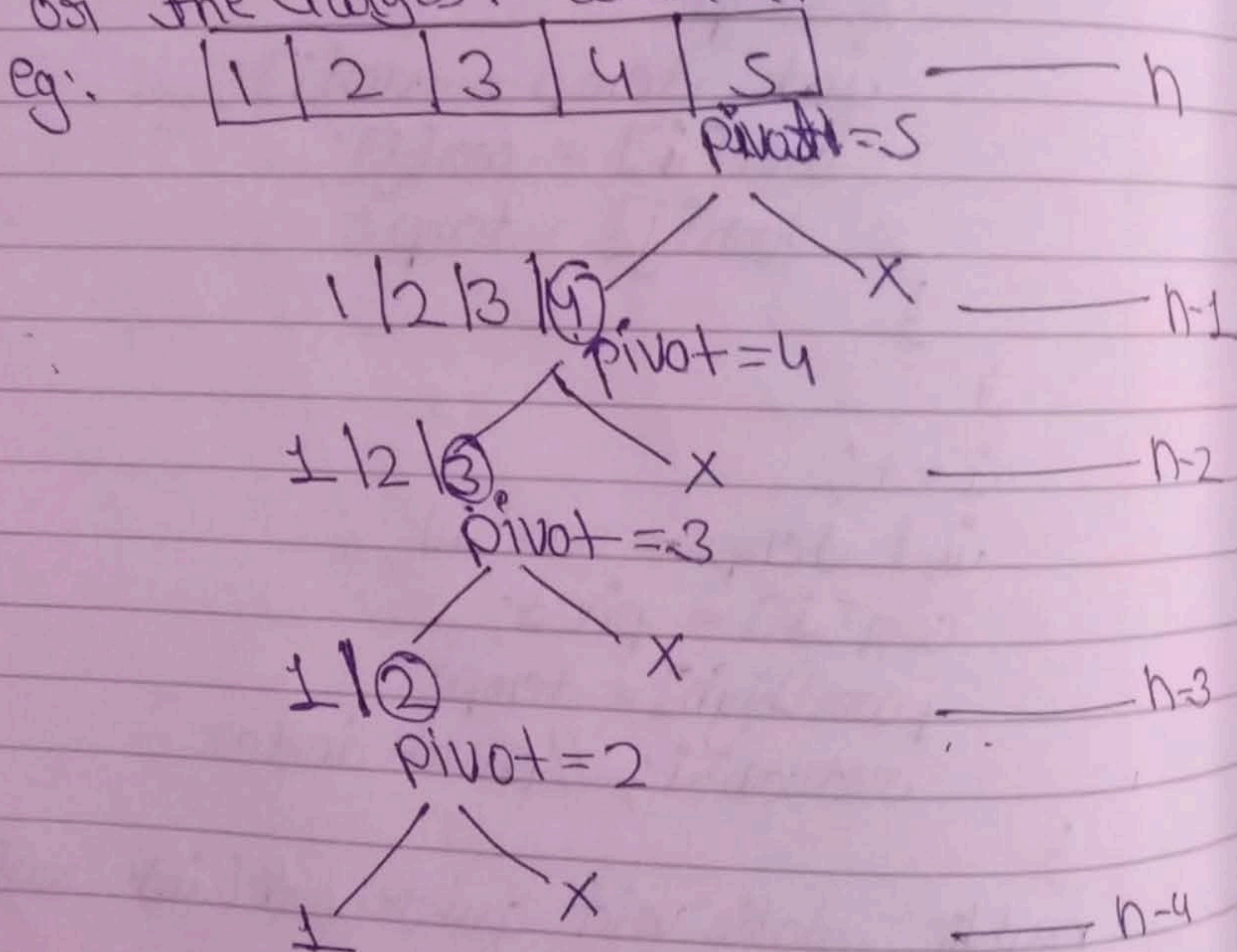
unstable

Time Complexity

Worst case: $O(n^2)$

Average case: $O(n \log n)$

→ Worst case occurs: when ~~any~~ array is completely sorted or completely unsorted (Pivot is always the smallest or the largest element)



$$T = \underbrace{n + n-1 + n-2 + n-3 + n-4 + \dots + 1}_{AP}$$

$$T = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} \approx n^2$$

Code

```
public class QuickSort
```

```
public static int partition(int arr[], int low,  
int high)
```

```
int i = low - 1
```

```
for (int j = low; j < high; j++)
```

```
if (arr[j] < pivot)
```

```
    i++;
```

```
    // swap.
```

```
    int temp = arr[i];
```

```
    arr[i] = arr[j];
```

```
    arr[j] = temp;
```

```
}
```

```
}
```

```
i++;
```

```
int temp = arr[i];
```

```
arr[i] = pivot;
```

```
arr[high] = temp;
```

```
return i; // pivot index
```

```
}
```

```
public static void quickSort(int arr[], int  
low, int high)
```

```
if (low < high)
```

```
int pidx = partition(arr, low, high);
```

```
quickSort(arr, low, pidx - 1);
```

```
quickSort(arr, pidx + 1, high);
```



```
public static void main(String args[]) {  
    int[] arr = {6, 3, 9, 5, 2, 8};  
    int n = arr.length;
```

```
    quickSort(arr, 0, n-1);
```

```
    for(int i=0; i<n; i++) {
```

```
        System.out.print(arr[i] + " ");
```

```
    }
```

```
    System.out.println();
```

```
}
```

```
}
```



```
QuickSort1.java > ...
1  import java.util.Scanner;
2
3  public class QuickSort1 {
4
5      static void quickSort(int[] arr, int low, int high) {
6          if (low < high) {
7              int pivotIndex = partition(arr, low, high);
8
9              quickSort(arr, low, pivotIndex - 1);
10             quickSort(arr, pivotIndex + 1, high);
11         }
12     }
13
14     static int partition(int[] arr, int low, int high) {
15         int pivot = arr[high];
16         int i = low - 1;
17
18         for (int j = low; j < high; j++) {
19             if (arr[j] <= pivot) {
20                 i++;
21
22                 int temp = arr[i];
23                 arr[i] = arr[j];
24                 arr[j] = temp;
25             }
26         }
27
28         int temp = arr[i + 1];
29         arr[i + 1] = arr[high];
30         arr[high] = temp;
31
32         return i + 1;
33     }
34 }
```

```
powershell >
PS C:\Users\Solv-IT\Desktop\java_module> java QuickSort
6
120 45 78 200 60 95
45 60 78 95 120 200
PS C:\Users\Solv-IT\Desktop\java_module>
```


QuickSort1.java > ...

```
3 public class QuickSort1 {
14 static int partition(int[] arr, int low, int high) {
30     arr[high] = temp;
31
32     return i + 1;
33 }
34
35 Run | Debug
36 public static void main(String[] args) {
37     Scanner sc = new Scanner(System.in);
38
39     int n = sc.nextInt();
40     int[] arr = new int[n];
41
42     for (int i = 0; i < n; i++) {
43         arr[i] = sc.nextInt();
44     }
45
46     quickSort(arr, low: 0, n - 1);
47
48     for (int i = 0; i < n; i++) {
49         System.out.print(arr[i] + " ");
50     }
51
52     sc.close();
53 }
54 }
55
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java QuickSort
6
120 45 78 200 60 95
45 60 78 95 120 200
PS C:\Users\Solv-IT\Desktop\java_module>
```


4. Quick Sort (Descending + Analysis)

Logic: Same as Quick Sort

Partition

$\text{arr}[j] \geq \text{pivot}$

Swap()

- Print Top 5 value
- Average of Top 5 : $\text{Top Average} = \text{Sum} / 5$
- Average = $\text{Total Sum} / N$
- Count: if (value > overall Average)
Count++

• Time Complexity

Sorting : $O(n \log n)$

Statistics : $O(n)$] + $O(n \log n)$

Code

```
import java.util.Scanner;
```

```
public class QuickSort {
```

```
    static void quickSort(long[] arr
```

```
        int low, int high) {
```

```
        if (low < high) {
```

```
            int pi = partition(arr, low, high);
```

```
            quickSort(arr, low, pi - 1);
```

```
            quickSort(arr, pi + 1, high);
```

```
        }  
    }  
    static int partition(long[] arr, int
```

```
        low, int high) {
```

```
        long pivot = arr[high];
```

```
        int i = low - 1;
```

```
        for (int j = low; j < high; j++) {
```

```
            if (arr[j] >= pivot) {
```

```
                i++;
```

```
                long temp = arr[i];
```

```
                arr[i] = arr[j];
```

```
                arr[j] = temp;
```



```

long temp = arr[i+1];
arr[i+1] = arr[high];
arr[high] = temp;
return i+1;

```

```

public static void main (String[] args) {

```

```

    Scanner sc = new Scanner (System.in);

```

```

    int n = sc.nextInt();

```

```

    long[] arr = new long[n];

```

```

    long sum = 0;

```

```

    for (int i = 0; i < n; i++) {

```

```

        arr[i] = sc.nextLong();

```

```

        sum += arr[i];
    }

```

```

    quickSort (arr, 0, n-1);

```

```

    // display sorted array

```

```

    for (int i = 0; i < n; i++) {

```

```

        System.out.println (arr[i] + " ");
    }

```

```

    System.out.println();

```

```

    // Top 5 values

```

```

    System.out.print ("Top 5: ");

```

```

    long topSum = 0;

```

```

    for (int i = 0; i < 5; i++) {

```

```

        System.out.print (arr[i] + " ");

```

```

        topSum += arr[i];
    }

```

```

    System.out.println();

```

```

    // Average of Top 5

```

```

    System.out.println ("Average of Top 5: " + (topSum/5));

```

```

    // Count values Above Overall avg

```

```

    double overallAverage = (double) sum/n;

```

```

    int count = 0;

```

```

    for (int i = 0; i < n; i++) {

```

```

        if (arr[i] > overallAverage) {

```

```

            count++;
        }
    }

```

```

    System.out.println ("Value Above Overall Average: " + count);
}

```

Input

8

500 1200 700 2000 900 1500 3000 2500

Output

3000 2500 2000 1500 1200 900 700 500

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040

Value Above Overall Average: 4

QuickSort.java > QuickSort > main(String[])

```
1 import java.util.Scanner;
2
3 public class QuickSort {
4
5     // Quick Sort in Descending Order
6     static void quickSort(long[] arr, int low, int high) {
7         if (low < high) {
8             int pi = partition(arr, low, high);
9
10            quickSort(arr, low, pi - 1);
11            quickSort(arr, pi + 1, high);
12        }
13    }
14
15    // Partition Function
16    static int partition(long[] arr, int low, int high) {
17        long pivot = arr[high];
18        int i = low - 1;
19
20        for (int j = low; j < high; j++) {
21            if (arr[j] >= pivot) { // Descending Order
22                i++;
23                long temp = arr[i];
24                arr[i] = arr[j];
25                arr[j] = temp;
26            }
27        }
28
29        long temp = arr[i + 1];
30        arr[i + 1] = arr[high];
31        arr[high] = temp;
32    }
33 }
```

powershell

```
PS C:\Users\Solv-IT\Desktop\java_module> java QuickSort
8
500 1200 700 2000 900 1500 3000 2500
Quick Sort Executed
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\Solv-IT\Desktop\java_module>
```


QuickSort.java / QuickSort / main(String[] args)

```
public class QuickSort {  
    static int partition(long[] arr, int low, int high) {  
  
        return i + 1;  
    }  
}
```

Run | Debug

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int n = sc.nextInt();  
    long[] arr = new long[n];
```

```
    long sum = 0;
```

```
    for (int i = 0; i < n; i++) {  
        arr[i] = sc.nextLong();  
        sum += arr[i];  
    }
```

```
    quickSort(arr, low: 0, n - 1);  
    System.out.println(x: "Quick Sort Executed");
```

```
    // 1. Display Sorted Array  
    for (int i = 0; i < n; i++) {  
        System.out.print(arr[i] + " ");  
    }  
    System.out.println();
```

```
    // 2. Top 5 Values  
    System.out.print(s: "Top 5: ");  
    long topSum = 0;
```

```
QuickSort.java:1: error: incompatible types: int cannot be converted to long  
    return i + 1;  
    ^  
QuickSort.java:1: error: incompatible types: int cannot be converted to long  
    return i + 1;  
    ^  
2 errors
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java QuickSort  
8  
500 1200 700 2000 900 1500 3000 2500  
Quick Sort Executed  
3000 2500 2000 1500 1200 900 700 500  
Top 5: 3000 2500 2000 1500 1200  
Average of Top 5: 2040  
Values Above Overall Average: 3  
PS C:\Users\Solv-IT\Desktop\java_module>
```



```

J Merge.java • J QuickSort.java X
J QuickSort.java > QuickSort > main(String[])
3  public class QuickSort {
36  public static void main(String[] args) {
37      System.out.println();
58
59      // 2. Top 5 Values
60      System.out.print(s: "Top 5: ");
61      long topSum = 0;
62      for (int i = 0; i < 5; i++) {
63          System.out.print(arr[i] + " ");
64          topSum += arr[i];
65      }
66      System.out.println();
67
68      // 3. Average of Top 5
69      System.out.println("Average of Top 5: " + (topSum / 5));
70
71      // 4. Count Values Above Overall Average
72      double overallAverage = (double) sum / n;
73
74      int count = 0;
75      for (int i = 0; i < n; i++) {
76          if (arr[i] > overallAverage) {
77              count++;
78          }
79      }
80
81      System.out.println("Values Above Overall Average: " + count);
82
83      sc.close();
84  }
85  }

```

```

powershell X
PS C:\Users\Solv-IT\Desktop\java_module> java QuickSort
8
500 1200 700 2000 900 1500 3000 2500
Quick Sort Executed
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\Solv-IT\Desktop\java_module>

```


5. Heap Sort

- Logic
1. Create Max Heap $\rightarrow$ Largest element becomes Root
 2. Swap Root with last element $\rightarrow$ Largest goes to last position
 3. Heapify remaining elements
 4. Repeat until size becomes 1.

Heap Formula

Left child = $2*i+1$

Right child = $2*i+2$

Time complexity

Build Heap : $O(n)$

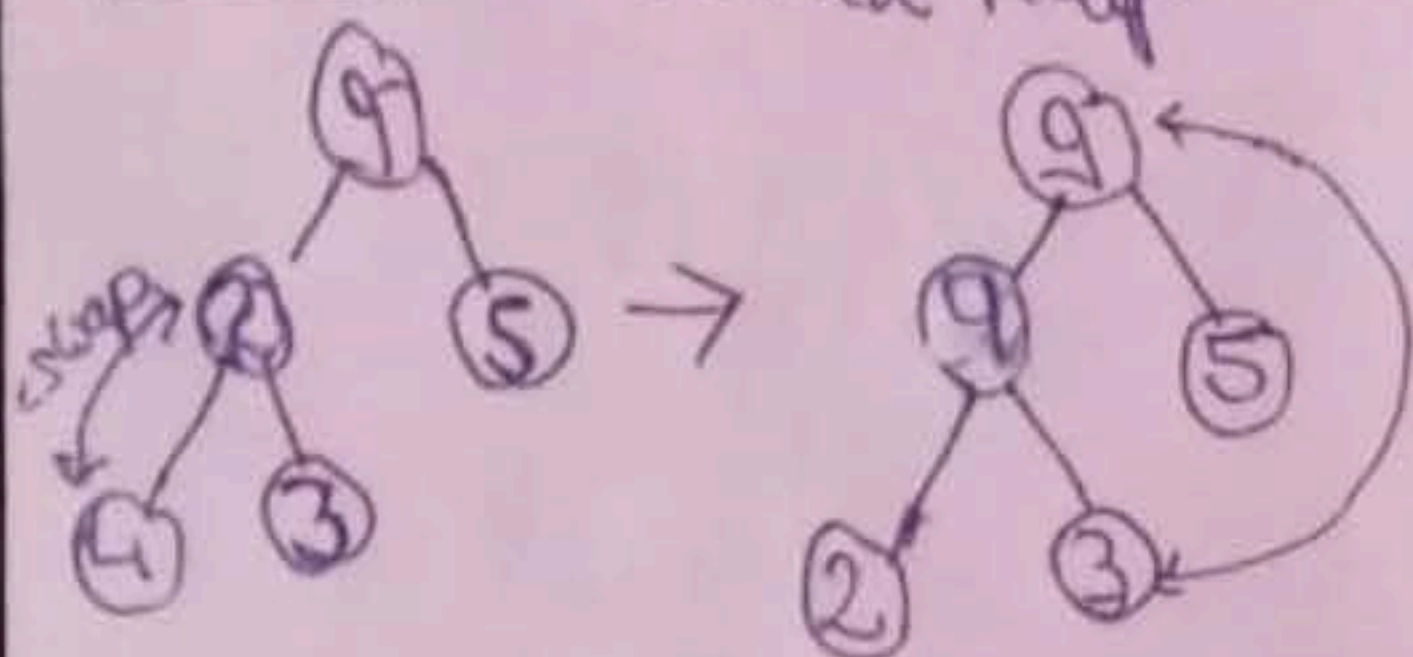
Sorting : $O(n \log n)$ $\rightarrow O(n \log n)$

Space : $O(1)$, unstable

eg:

9	2	5	4	3
---	---	---	---	---

Step 1: Create Max Heap



Step 1:

9	4	5	2	3
---	---	---	---	---

Step 2:

3	4	5	2	9
5	4	3	2	9

sorted

Step 3:

2	4	3	5	9
4	2	3	5	9

sorted

Step 4:

3	2	4	5	9
3	2	4	5	9

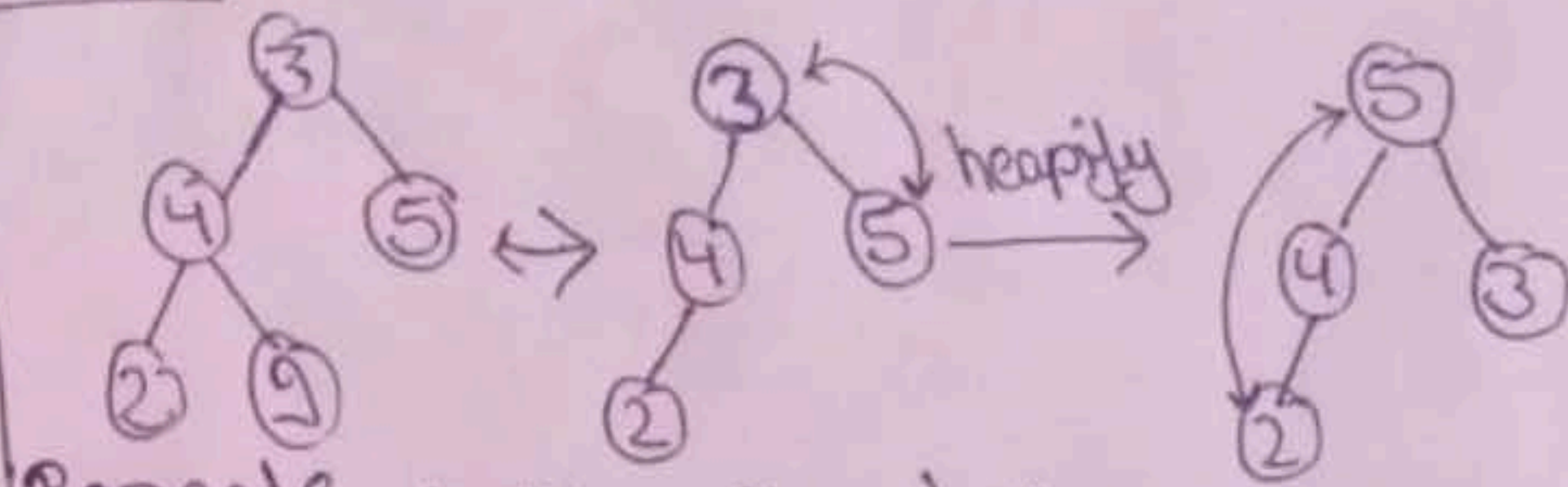
sorted

Step 5:

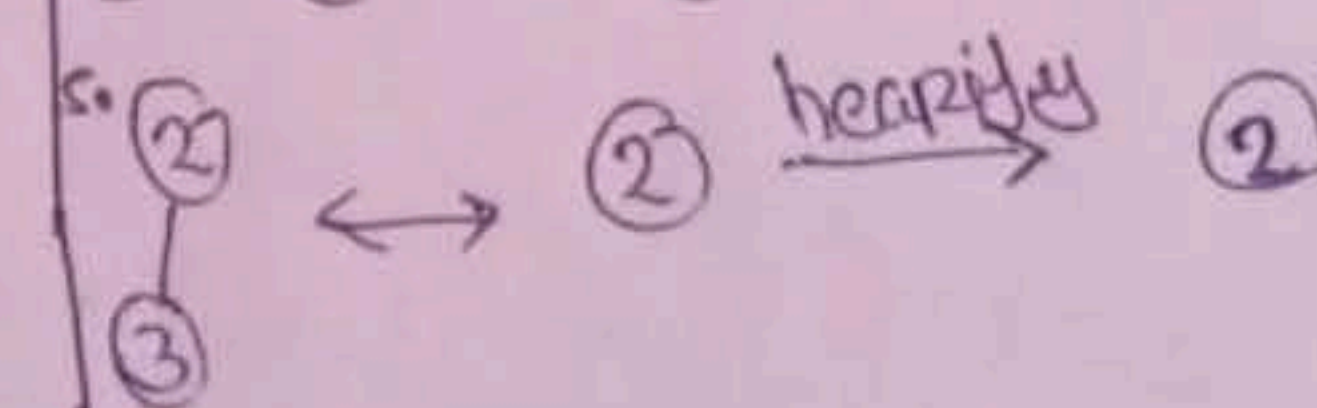
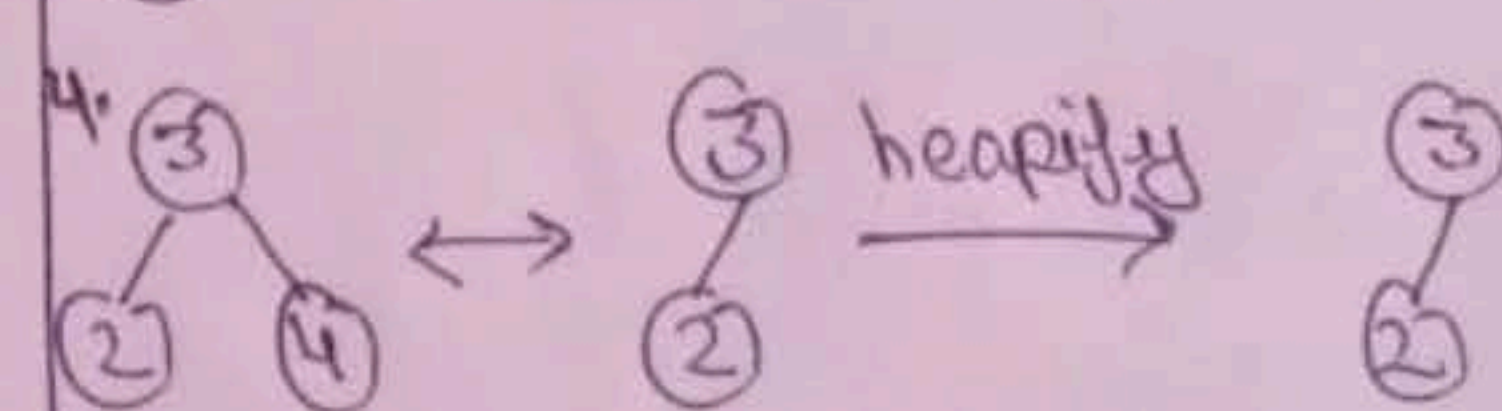
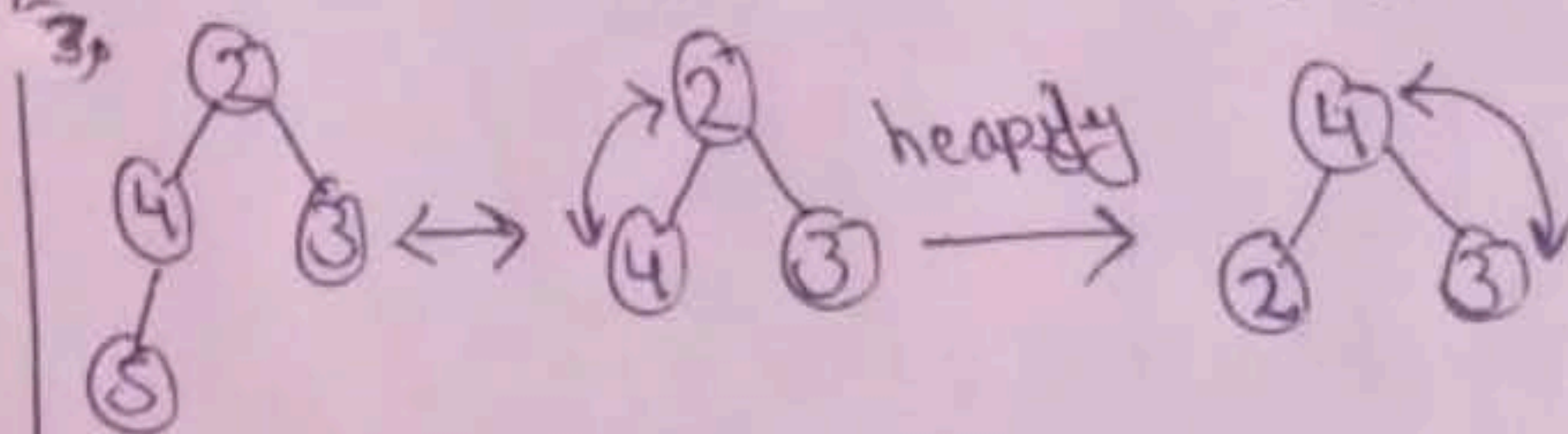
2	3	4	5	9
---	---	---	---	---

Sorted array

Step 2: Swap Root with last element & delete



Repeat until size becomes 1



6. Heap Sort with Hospital Analysis

```
import java.util.Scanner;
```

```
public class Heapsort {
```

```
    static void heapify(int[] arr, int n, int i) {
```

```
        int largest = i;
```

```
        int left = 2 * i + 1;
```

```
        int right = 2 * i + 2;
```

```
        if (left < n && arr[left] > arr[largest]) {
```

```
            largest = left;
```

```
        } if (right < n && arr[right] > arr[largest]) {
```

```
            largest = right;
```

```
        } if (largest != i) { swap(arr[i], arr[largest]);
```

```
            heapify(arr, n, largest);
```

```
        } static void heapsort(int[] arr, int n) {
```

```
            for (int i = n / 2 - 1; i >= 0; i--) {
```

```
                heapify(arr, n, i);
```

```
            } for (int i = n - 1; i > 0; i--) { swap(arr[0], arr[i];
```

```
                heapify(arr, i, 0);
```

```
            } } public static void main (String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        int[] arr = new int[n];
```

```
        double sum = 0;
```

```
        for (int i = 0; i < n; i++) {
```

```
            arr[i] = sc.nextInt();
```

```
            sum += arr[i];
```

```
        } heapsort(arr, n);
```

```
        for (int i = 0; i < n; i++) {
```

```
            System.out.print(arr[i] + " ");
```

```
        } System.out.println();
```

```
        System.out.println("Fastest: " + arr[0]);
```

```
        System.out.println("Slowest: " + arr[n-1]);
```

```
        double average = sum / n;
```

```
        System.out.print("Average: %.2f\n", average);
```

Logic

1. Read input

2. Build max heap

3. Sort array

4. Print sorted array

5. Find Fastest: arr[0]

6. Find Slowest: arr[n-1]

7. Average = sum / N

8. Count Faster than Average

if (arr[i] < average) {

count++

9. Percentage

(count * 100) / N

```
int count = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    if (arr[i] < average) {
```

```
        count++;
```

```
    } }
```

```
System.out.println("Cases
```

```
Faster Than Average: " +
```

```
count);
```

```
double percentage = (count * 100.0) / n;
```

```
System.out.print("Percentage
```

```
: %.2f%%\n", percentage);
```

```
sc.close();
```


HeapSort.java

HeapSort.java > HeapSort

```
1 import java.util.Scanner;
2
3 public class HeapSort {
4
5     static void heapify(int[] arr, int n, int i) {
6         int largest = i;
7         int left = 2 * i + 1;
8         int right = 2 * i + 2;
9
10        if (left < n && arr[left] > arr[largest]) {
11            largest = left;
12        }
13
14        if (right < n && arr[right] > arr[largest]) {
15            largest = right;
16        }
17
18        if (largest != i) {
19            int temp = arr[i];
20            arr[i] = arr[largest];
21            arr[largest] = temp;
22
23            heapify(arr, n, largest);
24        }
25    }
26
27    // Heap Sort
28    static void heapSort(int[] arr, int n) {
29
30        // Build Max Heap
31        for (int i = n / 2 - 1; i >= 0; i--) {
32            heapify(arr, n, i);
33        }
34    }
35}
```

powershell

```
PS C:\Users\Solv-IT\Desktop\java_module> javac HeapSort.java
PS C:\Users\Solv-IT\Desktop\java_module> java HeapSort
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500
PS C:\Users\Solv-IT\Desktop\java_module>
```



```

HeapSort.java > HeapSort
1  for (int i = n / 2 - 1; i >= 0; i--) {
2      heapify(arr, n, i);
3  }
4
5  // Extract elements one by one
6  for (int i = n - 1; i > 0; i--) {
7
8      int temp = arr[0];
9      arr[0] = arr[i];
10     arr[i] = temp;
11
12     heapify(arr, i, 0);
13 }
14 }

```

```

15
16 public static void main(String[] args) {
17     Run | Debug
18     Scanner sc = new Scanner(System.in);
19
20     int n = sc.nextInt();
21     int[] arr = new int[n];
22
23     for (int i = 0; i < n; i++) {
24         arr[i] = sc.nextInt();
25     }
26
27     heapSort(arr, n);
28
29     for (int i = 0; i < n; i++) {
30         System.out.print(arr[i] + " ");
31     }
32     sc.close();

```

```

• PS C:\Users\Solv-IT\Desktop\java_module> javac HeapSort.java
• PS C:\Users\Solv-IT\Desktop\java_module> java HeapSort
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500
• PS C:\Users\Solv-IT\Desktop\java_module> 

```


HeapSort.java

HeapSort.java > ...

```
1  import java.util.Scanner;
2
3  public class HeapSort {
4
5      // Heapify function
6      static void heapify(int[] arr, int n, int i) {
7          int largest = i;
8          int left = 2 * i + 1;
9          int right = 2 * i + 2;
10
11          if (left < n && arr[left] > arr[largest]) {
12              largest = left;
13          }
14
15          if (right < n && arr[right] > arr[largest]) {
16              largest = right;
17          }
18
19          if (largest != i) {
20              int temp = arr[i];
21              arr[i] = arr[largest];
22              arr[largest] = temp;
23
24              heapify(arr, n, largest);
25          }
26      }
27
28      // Heap Sort
29      static void heapSort(int[] arr, int n) {
30
31          // Build Max Heap
32          for (int i = n / 2 - 1; i >= 0; i--) {
```

powershell

PS C:\Users\Solv-IT\Desktop\java_module> java HeapSort

8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\Solv-IT\Desktop\java_module>

HeapSort.java > HeapSort > main(String[])

```
3 public class HeapSort {
46 public static void main(String[] args) {
60 heapSort(arr, n);
61 // Sorted array
62 for (int i = 0; i < n; i++) {
63     System.out.print(arr[i] + " ");
64 }
65 System.out.println();
66
67 // Fastest
68 System.out.println("Fastest: " + arr[0]);
69
70 // Slowest
71 System.out.println("Slowest: " + arr[n - 1]);
72
73 // Average
74 double average = sum / n;
75 System.out.printf("Average: %.2f%n", average);
76 format:
77 // Count faster than average
78 int count = 0;
79 for (int i = 0; i < n; i++) {
80     if (arr[i] < average) {
81         count++;
82     }
83 }
84 System.out.println("Cases Faster Than Average: " + count);
85 // Percentage
86 double percentage = (count * 100.0) / n;
87 System.out.printf("Percentage: %.2f%%n", percentage);
88
89 sc.close();
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\Solv-IT\Desktop\java_module> 
```


HeapSort.java > ...

```
3 public class HeapSort {
29 static void heapSort(int[] arr, int n) {
31     // Build Max Heap
32     for (int i = n / 2 - 1; i >= 0; i--) {
33         heapify(arr, n, i);
34     }
35
36     // Extract elements one by one
37     for (int i = n - 1; i > 0; i--) {
38         int temp = arr[0];
39         arr[0] = arr[i];
40         arr[i] = temp;
41
42         heapify(arr, i, 0);
43     }
44 }
45
```

Run | Debug

```
46 public static void main(String[] args) {
47
48     Scanner sc = new Scanner(System.in);
49
50     int n = sc.nextInt();
51     int[] arr = new int[n];
52
53     double sum = 0;
54
55     for (int i = 0; i < n; i++) {
56         arr[i] = sc.nextInt();
57         sum += arr[i];
58     }
59
```

```
PS C:\Users\Solv-IT\Desktop\java_module> java He
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\Solv-IT\Desktop\java_module> []
```